

# Call Workflows from Pages

Copy Page 

You can bind and trigger Workflows from [Pages Functions](#) by deploying a Workers project with your Workflow definition and then invoking that Worker using [service bindings](#) or a standard `fetch()` call.

## Note

You will need to deploy your Workflow as a standalone Workers project first before your Pages Function can call it. If you have not yet deployed a Workflow, refer to the Workflows [get started guide](#).

## Use Service Bindings

[Service Bindings](#) allow you to call a Worker from another Worker or a Pages Function without needing to expose it directly.

To do this, you will need to:

1. Deploy your Workflow in a Worker
2. Create a Service Binding to that Worker in your Pages project
3. Call the Worker remotely using the binding

For example, if you have a Worker called `workflows-starter`, you would create a new Service Binding in your Pages project as follows, ensuring that the service name matches the name of the Worker your Workflow is defined in:

**`{ }` wrangler.jsonc**  wrangler.toml

```
{
  "services": [
    {
      "binding": "WORKFLOW_SERVICE",
      "service": "workflows-starter"
    }
  ]
}
```

Your Worker can expose a specific method (or methods) that only other Workers or Pages Functions can call over the Service Binding.

In the following example, we expose a specific `createInstance` method that accepts our Payload and returns the [InstanceStatus](#) from the Workflows API:

**JS** JavaScript

TS TypeScript

index.js

```
import { WorkerEntrypoint } from "cloudflare:workers";

export default class WorkflowsService extends WorkerEntrypoint {
  // Currently, entrypoints without a named handler are not supported
  async fetch() {
    return new Response(null, { status: 404 });
  }

  async createInstance(payload) {
    let instance = await this.env.MY_WORKFLOW.create({
      params: payload,
    });

    return Response.json({
      id: instance.id,
      details: await instance.status(),
    });
  }
}
```

Your Pages Function would resemble the following:

**JS** JavaScript

TS TypeScript

[functions/request.js](#)

```
export const onRequest = async (context) => {  
  // This payload could be anything from within your app or from your frontend  
  let payload = { hello: "world" };  
  return context.env.WORKFLOWS_SERVICE.createInstance(payload);  
};
```

To learn more about binding to resources from Pages Functions, including how to bind via the Cloudflare dashboard, refer to the [bindings documentation for Pages Functions](#).

## Using fetch

### Service Bindings vs. fetch

We recommend using [Service Bindings](#) when calling a Worker in your own account.

Service Bindings don't require you to expose a public endpoint from your Worker, don't require you to configure authentication, and allow you to call methods on your Worker directly, avoiding the overhead of managing HTTP requests and responses.

An alternative to setting up a Service Binding is to call the Worker over HTTP by using the Workflows [Workers API](#) to create a new Workflow instance for each incoming HTTP call to the Worker:

**JS** JavaScript

## TS TypeScript

index.js

```
// This is in the same file as your Workflow definition
export default {
  async fetch(req, env) {
    let instance = await env.MY_WORKFLOW.create({
      params: payload,
    });
    return Response.json({
      id: instance.id,
      details: await instance.status(),
    });
  },
};
```

Your [Pages Function](#) can then make a regular fetch call to the Worker:

**JS** JavaScript

TS TypeScript

[functions/request.js](#)

```
export const onRequest = async (context) => {  
  // Other code  
  let payload = { hello: "world" };  
  const instanceStatus = await fetch("https://YOUR_WORKER.workers.dev/", {  
    method: "POST",  
    body: JSON.stringify(payload), // Send a payload for our Worker to pass to the Workflow  
  });  
  
  return Response.json(instanceStatus);  
};
```

You can also choose to authenticate these requests by passing a shared secret in a header and validating that in your Worker.

## Next steps

- Learn more about how to programmatically call and trigger Workflows from the [Workers API](#)
- Understand how to send [events and parameters](#) when triggering a Workflow
- Review the [Rules of Workflows](#) and best practices for writing Workflows

Was this helpful?



[Previous](#)

[← Rules of Workflows](#)

[Next](#)

[Metrics and analytics →](#)

Last updated: Feb 10, 2025

## Resources

[API](#)

[New to Cloudflare?](#)

[Products](#)

[Sponsorships](#)

[Open Source](#)

## Company

[cloudflare.com](#)

[Our team](#)

[Careers](#)

## Community

 [X](#)

 [Discord](#)

 [YouTube](#)

 [GitHub](#)

## Support

[Help Center](#)

[System Status](#)

[Compliance](#)

[GDPR](#)

## Tools

[Cloudflare Radar](#)

[Speed Test](#)

[Is BGP Safe Yet?](#)

[RPKI Toolkit](#)

[Certificate Transparency](#)